

Introducción a Docker

Contenedores, imágenes, volúmenes y Docker Compose

Por qué Docker

Durante el curso vas a trabajar con varios motores de bases de datos a la vez (relacional, documental, en memoria...) y con herramientas auxiliares que necesitan su propio servidor. Instalar todo eso directamente en tu ordenador — con las versiones correctas, sin que choquen entre sí, e igual en el instituto que en casa— es un dolor de cabeza.

Docker resuelve exactamente ese problema, y lo vamos a usar durante todo el módulo para levantar y gestionar el proyecto del curso.

¿Qué es Docker?

Docker es una herramienta que permite ejecutar aplicaciones dentro de contenedores: entornos aislados y ligeros que incluyen todo lo que la aplicación necesita para funcionar (código, dependencias, configuración) sin tocar el sistema operativo de tu máquina.

Metáfora: una caja estanca

Piensa en un contenedor como una caja sellada que lleva dentro exactamente lo que necesita para funcionar. No importa si tu ordenador tiene Windows, macOS o Linux, ni qué versión de Python o de PostgreSQL tengas instalada: dentro de la caja siempre es el mismo entorno.

El problema que resuelve

Sin Docker, instalar el entorno de un proyecto suele terminar así:

- “En mi ordenador funciona, en el tuyo no”, porque tenéis versiones distintas de la base de datos.
- Necesitas PostgreSQL para un proyecto y MongoDB para otro, con versiones incompatibles entre sí.
- Desinstalar un servicio deja archivos de configuración y datos sueltos por todo el sistema.
- Configurar un ordenador nuevo (o el del instituto) desde cero lleva media mañana.

Con Docker, cada servicio vive en su propio contenedor: se instala con un comando, se borra con otro, y no deja rastro en tu sistema si no quieres.

Docker vs. máquina virtual

Es habitual confundir contenedores con máquinas virtuales, pero no comparten recursos de la misma forma:



Máquina virtual

- Hardware
 - Hipervisor
 - SO invitado 1, SO invitado 2
 - Aplicación A, Aplicación B



Docker

- Hardware
 - Sistema operativo del host
 - Docker Engine
 - Contenedor A, Contenedor B

Cada máquina virtual carga un sistema operativo completo propio: pesada y lenta de arrancar. Los contenedores comparten el kernel del host y solo empaquetan la app y sus dependencias: arrancan en segundos.

Una idea equivocada muy común

Docker no sustituye a tu sistema operativo, ni monta una máquina virtual completa por debajo. Sigues teniendo un único Windows, macOS o Linux instalado; lo único que hace Docker es añadir una capa para ejecutar aplicaciones aisladas dentro de ese sistema.

Lo que Docker sustituye, en realidad, es la costumbre de instalar el programa directamente en tu máquina.

Conceptos clave: Imagen

Una imagen es un paquete de solo lectura con todo lo necesario para arrancar un programa: el sistema base, el software instalado y la configuración de fábrica.

postgres:18-alpine es una imagen: PostgreSQL en su versión 18, construida sobre una base Linux minimalista (alpine).

Una imagen no se modifica una vez creada: si necesitas cambiar algo, construyes una imagen nueva. Por eso se dice que es inmutable.

Conceptos clave: Contenedor

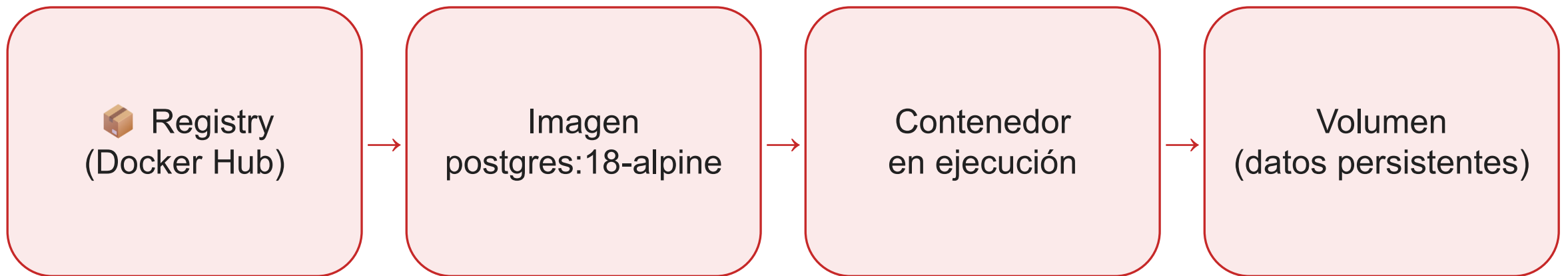
Un contenedor es lo que resulta de arrancar una imagen: un proceso en ejecución, aislado del resto de tu sistema, con su propio sistema de archivos y su propia red.

Puedes arrancar varios contenedores a partir de la misma imagen y no se pisarán entre sí, porque cada uno vive en su burbuja.

La imagen es como la receta de un pastel: un documento fijo que no cambia. El contenedor es el pastel que sale del horno al seguir esa receta.

De la imagen al volumen

El recorrido completo: de dónde sale una imagen, a dónde van a parar sus datos.



docker pull descarga la imagen del registry; docker run crea el contenedor; los datos que deben sobrevivir se guardan en un volumen aparte.

Conceptos clave: Volumen

Un contenedor está pensado para poder borrarse y recrearse sin drama: si una base de datos guarda sus ficheros solo dentro del contenedor, al eliminarlo pierdes todos los datos con él.

Un volumen es justo lo contrario: un espacio de almacenamiento que Docker gestiona por fuera del contenedor, así que sobrevive aunque el contenedor se borre y se vuelva a crear.

Es como guardar tus fotos en una nube en vez de en el propio ordenador: si el ordenador se estropea, las fotos siguen ahí.

Registry y puerto mapeado

Registry: de dónde salen las imágenes

Un servidor donde se publican y descargan imágenes, parecido a una tienda de aplicaciones. Docker Hub es el registry público más usado (postgres, mongo, redis...).

Puerto mapeado: cómo entrar en algo aislado

Por defecto, un contenedor está aislado de la red de tu ordenador. Un puerto mapeado asocia un puerto de tu máquina con el puerto donde escucha el servicio dentro del contenedor.

Conceptos clave: resumen

Concepto	En una frase
Imagen	Plantilla inmutable de la que se crean contenedores.
Contenedor	Una imagen puesta en marcha; puede haber varios a la vez.
Registry	Servidor donde se publican y descargan imágenes (Docker Hub).
Volumen	Almacenamiento fuera del contenedor que sobrevive a su borrado.
Puerto mapeado	Puerta entre un puerto de tu máquina y uno del contenedor.

Comandos básicos de Docker

No hace falta memorizarlos, pero conviene reconocerlos:

Comando	Qué hace
docker pull <imagen>	Descarga una imagen desde un registry (Docker Hub por defecto).
docker images	Lista las imágenes descargadas en tu máquina.
docker run <imagen>	Crea y arranca un contenedor a partir de una imagen.
docker ps	Lista los contenedores en ejecución (-a incluye los detenidos).
docker stop <contenedor>	Detiene un contenedor en ejecución sin borrarlo.
docker rm <contenedor>	Elimina un contenedor detenido.
docker logs <contenedor>	Muestra la salida (logs) de un contenedor.

Docker Compose

Un proyecto real casi nunca necesita un único contenedor: necesita una base de datos relacional, quizás una documental, una caché en memoria, un gestor de colas... Arrancar cada uno a mano con `docker run` y recordar todas sus opciones no escala.

Docker Compose resuelve esto con un fichero `docker-compose.yml`: describe todos los servicios en un único fichero declarativo, y Docker se encarga de crearlos, conectarlos y arrancarlos juntos con un solo comando.

Un docker-compose.yml sencillo

Para practicar no hace falta arrancar cinco servicios a la vez: el caso más simple posible tiene uno solo.

```
services:
  db:
    image: postgres:18-alpine
    environment:
      POSTGRES_USER: alumno
      POSTGRES_PASSWORD: alumno123
      POSTGRES_DB: prueba
    ports:
      - "5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Qué hace `docker compose up -d`

- Descarga la imagen postgres:18-alpine si no la tienes ya.
- Crea el contenedor db.
- Le pasa usuario, contraseña y nombre de base de datos por environment.
- Abre el puerto 5432 de tu máquina hacia el 5432 del contenedor.
- Guarda los datos en el volumen db_data, que sobrevive aunque el contenedor se recree.

Si ya tienes PostgreSQL instalado en tu máquina escuchando en el 5432, este contenedor chocará con él: cambia el primer número, por ejemplo a "5444:5432".

De un servicio a varios

Este ejemplo tiene un único servicio, así que no hace falta que hable con nadie más. En cuanto un proyecto necesita dos o más —por ejemplo, una aplicación web y su propia base de datos—, entran en juego dos cosas nuevas:

- Los contenedores se dirigen unos a otros por red usando el nombre del servicio como si fuera una dirección.
- `depends_on` decide en qué orden arrancan.

Vas a practicar exactamente esto en la Actividad 0.6, montando un WordPress completo con su base de datos.

Comandos básicos de Docker Compose

Comando	Qué hace
docker compose up -d	Crea y arranca todos los servicios definidos (-d en segundo plano).
docker compose ps	Lista los servicios del proyecto y su estado.
docker compose logs -f <servicio>	Muestra los logs de un servicio en tiempo real.
docker compose stop	Detiene todos los servicios sin eliminarlos.
docker compose down	Detiene y elimina contenedores y red. No borra volúmenes.
docker compose down -v	Igual que down, pero además elimina los volúmenes.

down -v borra los datos de verdad

`docker compose down -v` borra los datos de verdad, sin posibilidad de recuperarlos.

Para un reinicio normal, usa `down` (sin `-v`) o `stop`: los volúmenes con nombre se conservan y los datos siguen ahí la próxima vez que levantes el proyecto.

Ideas clave

- Docker ejecuta aplicaciones en contenedores: entornos aislados y ligeros que comparten el kernel del host.
- Una imagen es la plantilla inmutable; un contenedor es una instancia en ejecución de esa imagen.
- Un volumen guarda datos fuera del contenedor para que sobrevivan a su borrado.
- Docker Compose describe varios contenedores en un único `docker-compose.yml` y los levanta juntos.
- `services`, `ports`, `volumes`, `environment` y `depends_on` son las claves esenciales de un `docker-compose.yml`.
- `down` conserva los volúmenes; `down -v` también borra los datos.